complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + nonematch() negative validation

filter() null checks + nonematch() negative validation

filter() comparisons + nonematch() negative validation

filter() method calls returning boolean + nonematch() negative validation

distinct() rely on equals() / hashcode() + nonematch() negative validation

distinct() immutable objects + nonematch() negative validation

map() transform objects + nonematch() negative validation

map() create new objects + nonematch() negative validation

map() call pure functions + nonematch() negative validation

flatmap() return stream + nonematch() negative validation

flatmap() split data + nonematch() negative validation

flatmap() expand collections + nonematch() negative validation

sorted() natural ordering + nonematch() negative validation

sorted() custom comparator + nonematch() negative validation

sorted() multi-field compare + nonematch() negative validation

peek() logging + nonematch() negative validation

peek() debugging + nonematch() negative validation

peek() metrics + nonematch() negative validation

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **Terminal operation = noneMatch()**
- Purpose = **negative validation** (assert *no element* violates a rule)
- **Short-circuiting**: stops on the first violation
- Java **8+**, no collectors/reduce

---

1 filter() boolean checks + noneMatch() — Negative validation

```
import java.util.*;

public class FilterBooleanNoneMatch {
   public static void main(String[] args) {
      boolean ok = List.of(2, 4, 6, 8).stream()
            .filter(n -> n > 0)
            .noneMatch(n -> n % 2 != 0); // no odd numbers

      System.out.println(ok);
   }
}
```

---

2 filter() null checks + noneMatch() — Negative validation

```
import java.util.*;

public class FilterNullNoneMatch {
   public static void main(String[] args) {
      boolean ok = Arrays.asList("Anurag", "Amit").stream()
            .filter(Objects::nonNull)
            .noneMatch(String::isEmpty); // no empty strings

      System.out.println(ok);
   }
}
```

---

3 filter() comparisons + noneMatch() — Negative validation

```
import java.util.*;

public class FilterComparisonNoneMatch {
   public static void main(String[] args) {
      boolean ok = List.of(25, 30, 40).stream()
            .filter(age -> age >= 18)
            .noneMatch(age -> age > 60); // no seniors

      System.out.println(ok);
   }
}
```

---

4 filter() method calls returning boolean + noneMatch() — Negative validation

```
import java.util.*;

public class FilterMethodNoneMatch {

   static class User {
      boolean banned;
      User(boolean banned) { this.banned = banned; }
      boolean isBanned() { return banned; }
   }

   public static void main(String[] args) {
      boolean ok = List.of(new User(false), new User(false)).stream()
            .filter(u -> true)
            .noneMatch(User::isBanned); // no banned users

      System.out.println(ok);
   }
}
```

---

5 distinct() rely on equals() / hashCode() + noneMatch() — Negative validation

```
import java.util.*;

public class DistinctEqualsNoneMatch {

   static class Person {
      String email;
      Person(String email) { this.email = email; }
      @Override public boolean equals(Object o) {
         return o instanceof Person p && email.equals(p.email);
      }
      @Override public int hashCode() { return Objects.hash(email); }
   }

   public static void main(String[] args) {
      boolean ok = List.of(
            new Person("a@mail.com"),
            new Person("a@mail.com"),
            new Person("b@mail.com")
      ).stream()
       .distinct()
       .noneMatch(p -> p.email.endsWith("@spam.com"));

      System.out.println(ok);
   }
}
```

---

6 distinct() immutable objects + noneMatch() — Negative validation

```
import java.util.*;

public class DistinctImmutableNoneMatch {
   public static void main(String[] args) {
      boolean ok = List.of("Java", "Java", "Python").stream()
            .distinct()
            .noneMatch(lang -> lang.length() < 3);

      System.out.println(ok);
   }
}
```

---

7 map() transform objects + noneMatch() — Negative validation

```
import java.util.*;

public class MapTransformNoneMatch {
   public static void main(String[] args) {
      boolean ok = List.of("anurag", "amit").stream()
            .map(String::toUpperCase)
            .noneMatch(s -> s.contains(" ")); // no spaces

      System.out.println(ok);
   }
}
```

---

8 map() create new objects + noneMatch() — Negative validation

```
import java.util.*;

public class MapCreateObjectNoneMatch {

   static class UserDTO {
      String name;
      UserDTO(String name) { this.name = name; }
   }

   public static void main(String[] args) {
      boolean ok = List.of("Anurag", "Amit").stream()
            .map(UserDTO::new)
            .noneMatch(dto -> dto.name.isBlank());

      System.out.println(ok);
   }
}
```

---

9 map() call pure functions + noneMatch() — Negative validation

```
import java.util.*;

public class MapPureFunctionNoneMatch {

   static int normalize(int n) { return n * 10; }

   public static void main(String[] args) {
      boolean ok = List.of(1, 2, 3).stream()
            .map(MapPureFunctionNoneMatch::normalize)
            .noneMatch(n -> n <= 0);

      System.out.println(ok);
   }
}
```

---

10 flatMap() return Stream + noneMatch() — Negative validation

```
import java.util.*;

public class FlatMapStreamNoneMatch {
   public static void main(String[] args) {
      boolean ok = List.of(
            List.of(1, 2),
            List.of(3, 4)
      ).stream()
       .flatMap(List::stream)
       .noneMatch(n -> n < 0);

      System.out.println(ok);
   }
}
```

---

11 flatMap() split data + noneMatch() — Negative validation

```
import java.util.*;

public class FlatMapSplitNoneMatch {
   public static void main(String[] args) {
      boolean ok = List.of("Java Stream", "Flat Map").stream()
            .flatMap(s -> Arrays.stream(s.split(" ")))
            .noneMatch(word -> word.length() < 3);

      System.out.println(ok);
   }
}
```

---

12 flatMap() expand collections + noneMatch() — Negative validation

```
import java.util.*;

public class FlatMapExpandNoneMatch {
   public static void main(String[] args) {
      boolean ok = Map.of(
            "A", List.of("1", "2"),
            "B", List.of("3")
      ).values().stream()
       .flatMap(Collection::stream)
       .noneMatch(String::isEmpty);

      System.out.println(ok);
   }
}
```

---

13 sorted() natural ordering + noneMatch() — Negative validation

```
import java.util.*;

public class SortedNaturalNoneMatch {
   public static void main(String[] args) {
      boolean ok = List.of(1, 2, 3).stream()
            .sorted()
            .noneMatch(n -> n <= 0);

      System.out.println(ok);
   }
}
```

---

14 sorted() custom comparator + noneMatch() — Negative validation

```
import java.util.*;

public class SortedCustomNoneMatch {
   public static void main(String[] args) {
      boolean ok = List.of("Anurag", "Ravi", "Amit").stream()
            .sorted(Comparator.reverseOrder())
            .noneMatch(name -> name.length() < 4);

      System.out.println(ok);
   }
}
```

---

15 sorted() multi-field compare + noneMatch() — Negative validation

```
import java.util.*;

public class SortedMultiFieldNoneMatch {

   static class Employee {
      String name; int age;
      Employee(String name, int age) { this.name = name; this.age = age; }
   }

   public static void main(String[] args) {
      boolean ok = List.of(
            new Employee("Anurag", 30),
            new Employee("Anurag", 25),
            new Employee("Ravi", 35)
      ).stream()
       .sorted(Comparator.comparing((Employee e) -> e.name).thenComparingInt(e -> e.age))
       .noneMatch(e -> e.age < 18);

      System.out.println(ok);
   }
}
```

---

16 peek() logging + noneMatch() — Negative validation

```
import java.util.*;

public class PeekLoggingNoneMatch {
   public static void main(String[] args) {
      boolean ok = List.of(1, 2, 3).stream()
            .peek(n -> System.out.println("Checking: " + n))
            .noneMatch(n -> n < 0);

      System.out.println(ok);
   }
}
```

---

17 peek() debugging + noneMatch() — Negative validation

```
import java.util.*;

public class PeekDebugNoneMatch {
   public static void main(String[] args) {
      boolean ok = List.of(10, 20, 30).stream()
            .peek(n -> { if (n == 20) System.out.println("Debug hit"); })
            .noneMatch(n -> n < 10);

      System.out.println(ok);
   }
}
```

---

18 peek() metrics + noneMatch() — Negative validation

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsNoneMatch {
   public static void main(String[] args) {
      AtomicInteger seen = new AtomicInteger();

      boolean ok = List.of(5, 10, 15).stream()
            .peek(n -> seen.incrementAndGet())
            .noneMatch(n -> n <= 0);

      System.out.println("OK: " + ok);
      System.out.println("Processed: " + seen.get());
   }
}
```